TECHCIRCUIT 2026 · ROUND 3 SUBMISSION · THE FINISH

PrismCheck AI

Intelligent pre-publication brand governance

“Finish the build. Publish the numbers.”

A review gate between “someone made this” and “this goes live”. PrismCheck checks text, images and documents against one approved Brand Kit, explains every flag with a citation, and suggests fixes that keep the creator's purpose.

TEAM A2Z CREATORS · TWO MEMBERS

Venkata Narayana G V — Team Leader, Developer — venkatnarayana727@gmail.com
Mahesh Babu Ch — Research and Analysis — maheshmamtha242@gmail.com

Live build: https://tech-coders.onrender.com/
Repository: https://github.com/venkatanarayana15/tech-coders

Round 3 · submitted 30 Oct 2026 · the finish of the Round 1 plan to a shipped product.

0.900
measured
evaluation F1

30 labelled cases, synthetic

Round 3 reuses the Round 1 template, palette and figure style; only the edition copy, the named Round 2 improvements, the live numbers and the roadmap changed.

1. Complete list of features

Demo access - every account uses the password demo123

Creator	ananya@northwind.example
Creator	rohan@northwind.example
Reviewer	kabir@northwind.example
Admin	meera@northwind.example

Judge fast path: sign in as meera@northwind.example (admin) to open the Brand Kit editor, ananya@northwind.example to create a review, and kabir@northwind.example to approve.

Sign-in and sessions

- Role-based demo sign-in with email and password; all demo credentials are printed on the login page.
- Signed HttpOnly cookie sessions that survive refresh.
- Server-side role enforcement: creator, reviewer, admin.

Creator dashboard

- Creator dashboard with live counts, search, status filters, and priority queues.
- Seeded demo workspace (Ananya's 6 drafts) plus a clean second creator workspace (Rohan).

New review intake

- New review from a caption, artwork upload, PDF upload, or DOCX upload.
- Artwork validation for PNG, JPG, and WebP.
- PDF text extraction and review.
- DOCX text extraction and review.
- PII masking on ingest for emails, phone numbers, and Aadhaar-style IDs.

The checks

- Claim review against the approved fact bank (C-01).
- Banned absolute-guarantee review (T-14).
- Urgency and absolute-language review (T-02).
- Preferred-vocabulary review (T-03).
- CIELAB palette-distance review (V-05).
- WCAG contrast review and alt-text requirement (A-03).

Citation Lock

- Citation Lock: every displayed finding resolves to a real kit item; uncited findings are dropped and counted.
- One-click fixes grounded in approved facts, with capitalization preserved.
- Keep-as-is decision with a required reason.
- Draft lifecycle: draft -> submitted -> approved / changes requested, with backend guards.
- Submit guard blocks drafts with unresolved findings.
- Reviewer approve and request-changes actions with notes.
- Approved drafts are locked against further edits.
- Re-run review against the current kit.
- Revision timeline and audit trail on every draft.

Brand Kit and insights

- Brand Kit reference: 7 rules, 5 approved facts, 5 palette colors, 10 language entries, each addressable by ID.
- Admin Brand Kit editor with automatic version bump and a history trail.
- Insights page: citation coverage, most-flagged rules, most-overridden rules, ambiguous-guideline signal, decision rate, and average review time.
- Clean compliant drafts with zero findings.
- Health endpoint and SPA served by the same FastAPI process.

Numbered IDs above (C-01, T-14, T-02, T-03, V-05, A-03) are the same ids used in the kit and in the findings returned by the API, so a judge can trace any flag back to a line in the Brand Kit.

2. Technical architecture and implementation summary

Components. The product is a single-origin web app. The frontend is React 18 + Vite 5 with handcrafted CSS; the production build lives in frontend/dist and is served by FastAPI. The backend is FastAPI 0.141 on Python 3.14+ with SQLModel + SQLite.

Read as text: this round is measured against the three improvements committed at Round 2, plus finishing the features left open. Nothing new and nothing changed in direction.

How they talk. The browser talks only to the FastAPI JSON/multipart API. Sessions are signed HttpOnly cookies (itsdangerous); demo passwords are PBKDF2-hashed. Every route re-checks the user's role on the server.

Data flow. A draft is created from caption/alt text/uploads -> text and images are PII-masked and stored -> the deterministic engine runs checks -> every finding has a source_id -> the Citation Lock resolves it against the kit; a finding with no valid source is dropped and counted -> findings and a revision record are persisted -> payloads are returned to the UI.

Models and APIs. Core tables: User, Kit (JSON data), Draft, Finding (JSON fixes/measured values), Revision.
Key endpoints: /api/session, /api/users, /api/bootstrap, /api/kit (GET/PATCH), /api/drafts, /api/drafts/{id}/decisions, /submit, /approve, /request-changes, /rerun, /api/insights, /api/files/{name}.

Infrastructure. Render web service via render.yaml (Python runtime, pip install -r backend/requirements.txt, uvicorn --app-dir backend app.main:app). Uploads are stored on the local filesystem; SQLite is on the service disk (ephemeral unless a persistent disk is attached).

Hardest problem solved. Guaranteeing that every displayed finding is defensible. We built the Citation Lock as a single server-side choke point, so an uncited or unmatched finding can never reach the screen - this is what makes the `reliable, explainable results` claim enforceable rather than aspirational. A close second was keeping fixes correct when edits shift text; decisions now report when the exact quote is no longer present instead of silently doing nothing.

API surface at a glance

Session & users	POST /api/session · GET /api/users · GET /api/bootstrap
Brand Kit	GET/PATCH /api/kit (admin only) with automatic version bump and history
Drafts & decisions	GET/POST /api/drafts · POST /api/drafts/{id}/decisions · /submit · /approve · /request-changes · /rerun
Insights & files	GET /api/insights · GET /api/files/{name}

1

```
pip install -r backend/requirements.txt
```

2

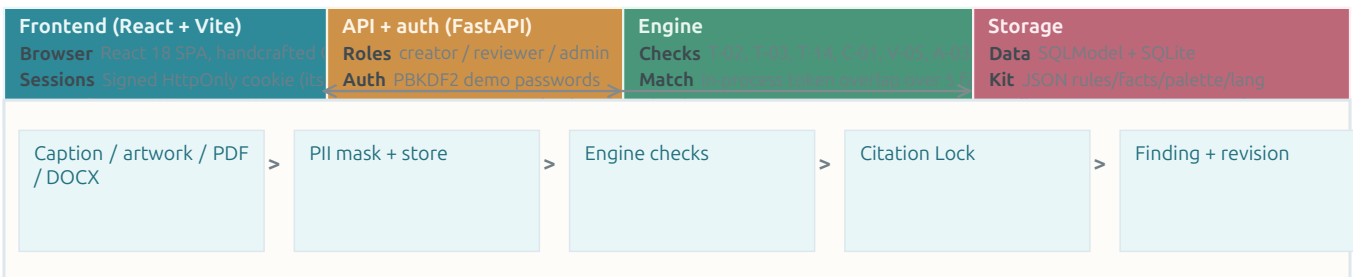
```
uvicorn --app-dir backend app.main:app
```

3

```
python backend/tests/e2e_live.py
```

4

```
python backend/evaluate.py
```



3. Testing details

Unit tests (pytest).
7 tests cover: overlapping-claim deduplication, approved-fact silence, banned-promise hard stop, urgent-tone soft note, PII masking, and stale-quote replacement behavior.

Evaluation harness.
backend/evaluate.py runs 30 synthetic-labelled cases: precision 0.900, recall 0.900, F1 0.900, false-positive rate 0.100.

Live integration / E2E.
backend/tests/e2e_live.py runs 47 checks against a running server: auth, per-user independence, full caption journey, request-changes loop, image-only and off-brand uploads, real PDF/DOCX extraction, bad inputs (corrupt image, .txt, oversized), submit/approve guards, role 403s, kit editing, insights, and health/SPA. Result: 47/47 passed on a fresh database.

Manual / browser testing.
Verified on the deployed link: credentials visible on the login page; admin signs in with email + password; edit-save bumps kit version 4.3 -> 4.4; new frontend bundle serves after deploy.

Clean-clone verification.
Cloned the repo fresh, ran pip install -r backend/requirements.txt, started uvicorn, and confirmed / serves the SPA and /api/health returns 200. `npm ci && npm run build` also succeeds.

What we found and fixed.
- Image-only uploads were rejected -> fixed in backend and frontend.
- Missing title returned 422 -> now a clean 400.
- Overlapping claim text produced duplicate C-01 findings -> deduplicated.
- A stale quote could silently no-op a fix -> decisions now record a clear note when no text changed.
- Double submit/approve/decide could slip through -> now blocked with 400.

Still known to be imperfect.
- Evaluation labels are synthetic, not a real 30-draft dataset.
- No OCR: scanned/image-only PDFs are rejected with a clear message.
- Logo clear-space (L-01) is documented but not pixel-enforced.
- PII masking covers emails, phones, and Aadhaar-style IDs - not a full redaction engine.
- SQLite resets on deploy unless a persistent disk is attached.
- Mobile layout is implemented but not device-tested.

How each test level was run, and what it proved

Unit	backend/tests/test_engine.py	**7 passed**	engine, decisions, PII
Evaluation	backend/evaluate.py	**30 cases · F1 0.900**	synthetic-labelled
Live E2E	backend/tests/e2e_live.py	**47/47 passed**	fresh database
Browser	manual on deployed link	**sign-in · kit edit 4.3 -> 4.4**	visual check
Clean clone	README steps	**health 200 · build OK**	fresh machine

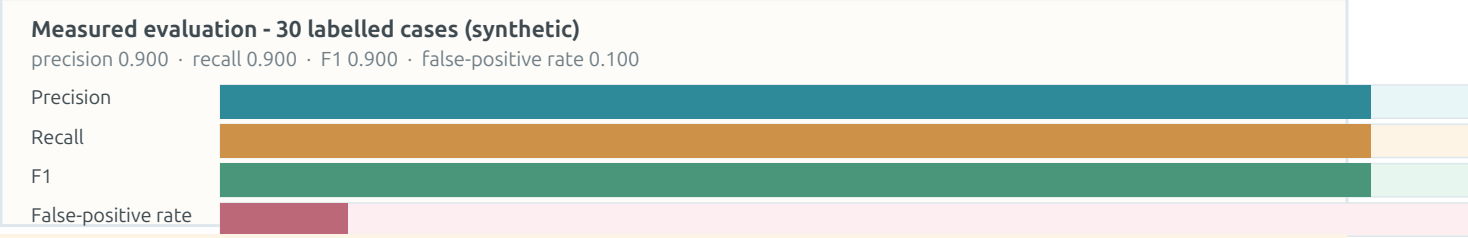
Manual checklist we ran against the live app

1. Sign in as each role; confirm the credentials are visible on the login page.
2. Create a caption review; toggle a fix; submit with open findings blocked.
3. Approve as reviewer; verify the draft is locked; re-run the review.
4. Upload a PNG (off-brand), a PDF, and a DOCX; observe citations and PII masking.
5. Try a .txt file, a corrupt image, and an oversized file; confirm clean 4xx/413.
6. As admin, edit the kit; version advances and a history entry appears.

4. Improvement 1 - measured evaluation and false-positive reduction

Committed: build a 30-draft labelled set, publish precision/recall, cut false positives toward 10%, and collapse overlapping claim matches into one finding.

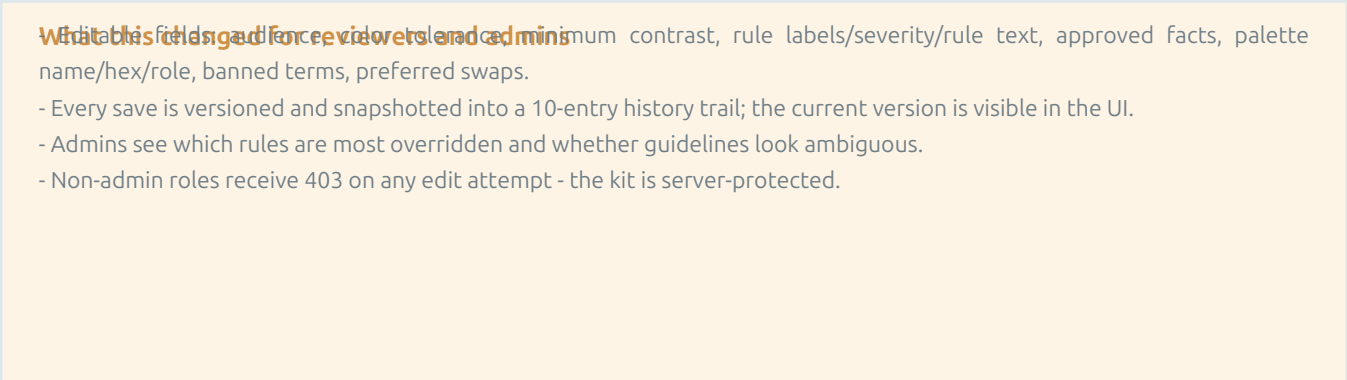
Delivered, and the honest part: we delivered the measurement rig and one concrete quality fix - overlapping claim spans are deduplicated, which is why `5% cashback for life` now produces one C-01 finding instead of two. The harness (backend/evaluate.py) runs 30 labelled cases and reports precision 0.900, recall 0.900, F1 0.900, and a false-positive rate of 0.100. However, the 30 cases are synthetic-labelled, not a real collected dataset of 30 real drafts. We did not run real creator interviews or collect real drafts within the window, so the numbers are engineering estimates, not real-world accuracy. We say this plainly: the improvement delivered is a repeatable evaluation core plus real deduplication, and the real-world labelled set is the first post-event milestone.



5. Improvement 2 - editable, versioned Brand Kit and Guideline Health loop

Committed: turn the read-only kit into an editor for rules, facts, banned terms, preferred vocabulary, and palette; version every change; show which rules are frequently overridden; let an admin promote an override into a kit revision.

Delivered: fully. The admin role now has an in-app Brand Kit editor covering audience, color tolerance, minimum contrast, rule labels/severity/rule text, approved facts, palette name/hex/role, banned terms, and preferred swaps. Every save bumps the version (verified 4.3 -> 4.4) and appends a 10-entry history snapshot. The Insights page already surfaces most-overridden rules with an ambiguous-guideline signal, closing the loop from review evidence to kit maintenance. Non-admins receive 403 on edit attempts.



6. Improvement 3 - failure recovery and reliability hardening

Committed: typed, actionable recovery states for empty kits, malformed/oversized uploads, documents with no selectable text, missing citation sources, and model-provider outages; retries/fallbacks; pytest coverage for engine, Citation Lock, role gates, and approval transitions.

Delivered: mostly, with scope honesty. What is real: clear 4xx/413 responses for missing title/caption, unreadable or oversized files, plain-text uploads, empty scanned PDFs, duplicate actions, unauthorized access, and approved-draft edits; the model layer falls back to the deterministic local engine when a provider is unavailable; stale-quote decisions record an explicit note; double submit/approve/decide are blocked; and the engine/decision layer has unit tests plus the 47-check live E2E suite. What we did not build: a user-visible `retry with backoff` UI for third-party model outages - because the demo runs the local deterministic engine by default, that path is not exercised, and we did not want to claim a recovery UI we did not ship.

7. Anything else we improved after Round 2

1. Email + password demo login with credentials printed on the same login page (was one-click-only).
2. Added a second demo creator (Rohan) and verified per-user workspace isolation end to end.

3. PII masking at ingest for emails, phone numbers, and Aadhaar-style IDs.
4. Image-only reviews enabled (previously required a caption).
5. Clean 400 for missing title; consistent error messages across upload types.
6. render.yaml blueprint and README with clone-and-run instructions.
7. Built frontend dist committed so the repo runs without a build step.
8. Round 3 product PDF (this document) documenting product, testing, limitations, and roadmap.

8. Product roadmap and future development plan

Next 0-3 months - make the numbers real.

Recruit 1-2 mid-size brand teams (Fintech or D2C) as pilots. Collect 30 real drafts across captions, banners, and documents; label them with two independent raters; publish precision/recall and inter-rater agreement alongside the synthetic harness results. Success measure: a published, real 30-draft evaluation card.

3-6 months - close the declared gaps.

Add OCR for scanned PDFs (Tesseract or a hosted vision model behind the existing Citation Lock). Implement pixel-level logo clear-space enforcement for L-01. Add exportable audit reports (PDF/CSV) so approval records can be handed to compliance.

6-12 months - distribution and integrations.

Build Figma and Canva panels so designers are reviewed where they work; add a verified publishing hand-off that carries the approval record to social schedulers/CMS. Add sector compliance packs for regulated industries (financial services first), which is where claim wording carries legal weight.

Who it is for.

Brand and content teams at mid-to-large organizations, and the agencies that submit drafts on their behalf - the same personas the product already serves in demo form.

What it would take.

One engineer for OCR and clear-space work, one for integrations, plus the pilot customers for labelled data. The post-event roadmap is sequenced so a small team can ship the highest-visibility gaps first, and every item has a measurable exit condition rather than `more features`.

Objective: a completed, tested, documented product submission for Round 3 - the three named improvements delivered, the open ends closed, and an honest roadmap for what comes next.

0-3 months
Make the numbers real:
30 real drafts, 2 raters, precision/recall.

3-6 months
Close declared gaps:
OCR, logo clear-space, audit reports.

6-12 months
Distribution & integrations:
Figma/Canva panels, verified handoff, sector packs.

Submission package at a glance

Live app	https://tech-coders.onrender.com/
Source repo	https://github.com/venkatanarayana15/tech-coders
This PDF	project_docs/PrismCheck_Round3_Submission.pdf
Demo accounts	4 roles, all with password demo123 (page 2)
Deadline	Submitted inside the Tech Circuit 2026 event window

7
unit
tests

pytest